# Mastering Spring Boot Batch Processing & Scheduling -- Internals, Architecture, and Enterprise Usage Guide

---

**Author:** Senior Java Architect **Target Audience:** Java developers with 3+ years of experience aiming for senior/architect-level roles **Prerequisites:** Core Java, Spring Boot, Spring Data JPA, REST APIs, Microservices basics

---

# Table of Contents


1. Introduction to Batch Processing
2. Spring Boot Scheduler
3. Spring Scheduling Architecture
4. Spring Boot Scheduler Implementation
5. Cron Expressions
6. Spring Boot Batch Processing
7. Spring Batch Architecture
8. Spring Batch Components
9. Spring Batch Project Structure
10. Complete Batch Job Implementation
11. Real-Time Production Scenario
12. Handling Large Datasets
13. Error Handling in Batch Jobs
14. Job Monitoring
15. Spring Batch Database Tables
16. Spring Batch + Scheduler Integration
17. Microservice Architecture with Batch
18. Production Best Practices
19. Performance Optimization
20. Common Production Issues
21. Interview Questions (50+)
22. Coding Exercises (15+)


---

# Part 1: Introduction to Batch Processing

## 1.1 What is Batch Processing?

**Batch processing** is the execution of a series of tasks (jobs) on a collection of data without manual intervention. Jobs run at scheduled times or when triggered, processing bulk data in a single operation rather than handling individual records one at a time.

Key characteristics:

- **Non-interactive** -- no user input required during execution
- **Volume-oriented** -- processes large datasets (thousands to millions of records)
- **Scheduled** -- runs at specific times (nightly, hourly, weekly)
- **Automated** -- no human intervention after trigger
- **Transactional** -- supports rollback on failure

## 1.2 Why Batch Processing is Required in Enterprise Systems

| Requirement | Why Batch Processing? |
|---|---|
| Monthly payroll for 50,000 employees | Cannot process in real-time; requires bulk salary calculation |
| Daily bank statement generation | Millions of accounts need statements generated overnight |
| Government benefits eligibility | Nightly recalculation for eligibility changes |
| Insurance premium recalculation | Quarterly bulk recalculation for all policyholders |
| Data warehouse ETL | Extract, transform, load millions of records from source systems |
| Regulatory reporting | Generate compliance reports from aggregated data |

## 1.3 Batch vs Real-Time vs Event-Driven Processing

| Feature | Batch Processing | Real-Time Processing | Event-Driven Processing |
|---|---|---|---|
| **When it runs** | Scheduled (cron, timer) | Immediately on request | On event occurrence |
| **Data volume** | Large (bulk) | Single record | Single event |
| **Latency** | Minutes to hours | Milliseconds | Milliseconds to seconds |

| | | | |
|---|---|---|---|
| **Example** | Nightly payroll | REST API response | Kafka consumer |
| **User waits?** | No | Yes | No |
| **Tools** | Spring Batch, Quartz | Spring MVC, WebFlux | Kafka, RabbitMQ |
| **Use case** | Report generation | Order placement | Order status notification |

## 1.4 Real-World Enterprise Examples

### Bank Transaction Processing

```
Nightly Job (2:00 AM):
1. Read all transactions from the day
2. Calculate interest for savings accounts
3. Apply fees and charges
4. Generate account statements
5. Update account balances
6. Generate regulatory reports
```

### Government Benefits System

```
Daily Eligibility Job (12:00 AM):
1. Read all citizen records with status changes
2. Verify income thresholds
3. Check document validity
4. Calculate benefit amounts
5. Generate eligibility notices
6. Queue notices for mailing
```

### Invoice Generation System

```
Monthly Job (1st of every month):
1. Read all customer accounts
2. Calculate usage charges
3. Apply discounts and taxes
4. Generate PDF invoices
5. Store in document management system
6. Send email notifications
```

---

# Part 2: Spring Boot Scheduler

## 2.1 What is Scheduling?

**Scheduling** is the mechanism to execute a task automatically at a specific time, at fixed intervals, or based on a cron expression. Spring Boot provides a lightweight, annotation-based scheduling framework built into the core.

## 2.2 Why Scheduling is Used

| Scenario | Scheduling Need |
|---|---|
| Cache refresh | Refresh distributed cache every 15 minutes |
| Database cleanup | Delete expired sessions every hour |
| Report generation | Generate daily sales report at midnight |
| Health checks | Ping dependent services every 30 seconds |
| Notification emails | Send digest emails every morning at 8 AM |
| Data synchronization | Sync data with external API every 5 minutes |
| Log archival | Archive and compress old logs weekly |
| Reconciliation | Match payment records with bank at end of day |

## 2.3 Common Production Jobs

### Daily Report Generation

```java
@Scheduled(cron = "0 0 6 * * *") // Every day at 6:00 AM
public void generateDailySalesReport() {
    LocalDate yesterday = LocalDate.now().minusDays(1);
    List<Order> orders = orderRepository.findByDate(yesterday);
    ReportData report = reportService.buildSalesReport(orders);
    pdfService.generate(report);
    emailService.sendToManagement(report);
}
```

### Cache Refresh

```java
@Scheduled(fixedRate = 900000) // Every 15 minutes
public void refreshProductCache() {
    List<Product> products = productRepository.findAllActive();
    cacheManager.getCache("products").clear();
    products.forEach(p -> cacheManager.getCache("products").put(p.getId(), p));
    log.info("Product cache refreshed with {} items", products.size());
}
```

### Database Cleanup

```java
@Scheduled(cron = "0 0 3 * * *") // Every day at 3:00 AM
public void cleanupExpiredSessions() {
    Instant cutoff = Instant.now().minus(Duration.ofHours(24));
    int deleted = sessionRepository.deleteByLastAccessBefore(cutoff);
    log.info("Cleaned up {} expired sessions", deleted);
}
```

---

# Part 3: Spring Scheduling Architecture

## 3.1 Architecture Overview

```
+-------------------------------------------------------------+
|                  Spring Boot Application                    |
|                                                             |
|  +------------------+    +-------------------------------+  |
|  |   @Scheduled     |    |     TaskScheduler             |  |
|  |   Methods         |--->|   (ThreadPoolTaskScheduler)   |  |
|  +------------------+    +-------------------------------+  |
|         |                          |                        |
|         v                          v                        |
|  +------------------+    +-------------------------------+  |
|  |  Service Layer   |    |   ScheduledExecutorService    |  |
|  |  (Business Logic)|    |   (Thread Pool: 1 by default) |  |
|  +------------------+    +-------------------------------+  |
|         |                                                   |
|         v                                                   |
|  +------------------+                                        |
|  |   Repository     |                                        |
|  |   (Data Access)  |                                        |
|  +------------------+                                        |
|         |                                                   |
|         v                                                   |
|  +------------------+                                        |
|  |   Database       |                                        |
|  +------------------+                                        |
+-------------------------------------------------------------+
```

## 3.2 How @Scheduled Works Internally

1. **Application Startup**: Spring scans for `@EnableScheduling` annotation
2. **Bean Post Processing**: `ScheduledAnnotationBeanPostProcessor` finds all `@Scheduled` methods
3. **Task Registration**: Each method is registered as a `ScheduledTask` with the `TaskScheduler`
4. **Thread Pool**: By default, Spring uses a **single-threaded** `ScheduledExecutorService`
5. **Execution**: At trigger time, the executor runs the method in its thread pool

6. **Repeat**: The scheduler re-schedules based on fixedRate, fixedDelay, or cron

**WARNING:** Default thread pool size is **1**. If you have multiple scheduled tasks, they will execute sequentially, not in parallel!

## Configuring Thread Pool

```java
@Configuration
public class SchedulerConfig implements SchedulingConfigurer {

    @Override
    public void configureTasks(ScheduledTaskRegistrar registrar) {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5); // 5 threads for scheduled tasks
        scheduler.setThreadNamePrefix("scheduler-");
        scheduler.setErrorHandler(t ->
            log.error("Scheduled task error", t));
        scheduler.initialize();
        registrar.setTaskScheduler(scheduler);
    }
}
```

---

# Part 4: Spring Boot Scheduler Implementation

## 4.1 Enable Scheduling

```
@SpringBootApplication
@EnableScheduling
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

## 4.2 @Scheduled Types

### fixedRate -- Execute at Fixed Intervals

**What:** Executes the method at a fixed interval, measured from the **start** of the previous execution.

**When to use:** When you need consistent execution frequency regardless of task duration.

**Production Example:** Health check pings

```
@Component
public class HealthCheckScheduler {

    @Autowired
    private List<ServiceHealthCheck> healthChecks;

    @Scheduled(fixedRate = 30000) // Every 30 seconds
    public void checkServiceHealth() {
        healthChecks.forEach(check -> {
            try {
                boolean healthy = check.ping();
                metricsService.recordHealth(check.getName(), healthy);
                if (!healthy) {
                    alertService.sendAlert(check.getName() + " is DOWN");
                }
            } catch (Exception e) {
                log.error("Health check failed for {}", check.getName(), e);
```

```
            }
        });
    }
}
```

## fixedDelay -- Execute After Previous Completes

**What:** Waits for the previous execution to **complete**, then waits the specified delay before starting the next.

**When to use:** When tasks must not overlap, and the next run depends on the previous one finishing.

**Production Example:** Data synchronization

```java
@Component
public class DataSyncScheduler {

    @Scheduled(fixedDelay = 60000) // 60 seconds after previous completes
    public void syncWithExternalSystem() {
        log.info("Starting sync at {}", Instant.now());
        List<Record> unsyncedRecords = recordRepository.findUnsynced();

        for (Record record : unsyncedRecords) {
            try {
                externalApi.push(record);
                record.setSynced(true);
                recordRepository.save(record);
            } catch (Exception e) {
                log.error("Sync failed for record {}", record.getId(), e);
            }
        }
        log.info("Sync completed. Processed {} records",
unsyncedRecords.size());
    }
}
```

## cron -- Execute Based on Cron Expression

**What:** Executes at times defined by a cron expression (most flexible).

**When to use:** Specific times of day, specific days of week/month.

**Production Example:** End-of-day reconciliation

```java
@Component
public class ReconciliationScheduler {

    @Scheduled(cron = "0 0 23 * * MON-FRI") // 11 PM on weekdays
    public void runDailyReconciliation() {
        LocalDate today = LocalDate.now();
        log.info("Starting reconciliation for {}", today);

        List<Transaction> ourRecords = transactionRepo.findByDate(today);
        List<BankTransaction> bankRecords = bankApi.getTransactions(today);

        ReconciliationResult result = reconciliationService
            .reconcile(ourRecords, bankRecords);

        reportService.generateReconciliationReport(result);

        if (result.hasDiscrepancies()) {
            alertService.notifyFinanceTeam(result.getDiscrepancies());
        }
    }
}
```

## Comparison

| Feature | fixedRate | fixedDelay | cron |
|---|---|---|---|
| Timer starts from | Start of previous | End of previous | Absolute time |
| Overlap possible? | Yes (if task is slow) | No | Depends |
| Best for | Periodic polling | Sequential processing | Specific schedules |
| Example | Health checks | Data sync | Nightly reports |

---

# Part 5: Cron Expressions

## 5.1 Cron Syntax (Spring Format)

Spring uses a **6-field** cron expression:

```
+------------ Second (0-59)
| +---------- Minute (0-59)
| | +-------- Hour (0-23)
| | | +------ Day of Month (1-31)
| | | | +---- Month (1-12 or JAN-DEC)
| | | | | +-- Day of Week (0-7 or SUN-SAT, 0 and 7 are both Sunday)
| | | | | |
* * * * * *
```

## 5.2 Special Characters

| Character | Meaning | Example |
|---|---|---|
| `*` | Every value | `* * * * * *` = every second |
| `,` | List of values | `0 0 8,12,18 * * *` = at 8AM, 12PM, 6PM |
| `-` | Range | `0 0 9-17 * * *` = every hour 9AM-5PM |
| `/` | Increment | `0 */15 * * * *` = every 15 minutes |
| `?` | No specific value | Used in Day-of-Month or Day-of-Week |
| `MON-FRI` | Named days | `0 0 9 * * MON-FRI` = weekdays at 9AM |

## 5.3 Common Cron Expressions

| Cron Expression | Description |
|---|---|
| `0 * * * * *` | Every minute |
| `0 0 * * * *` | Every hour |
| `0 0 0 * * *` | Every midnight |

| | |
|---|---|
| `0 0 6 * * *` | Every day at 6:00 AM |
| `0 0 0 * * SUN` | Every Sunday at midnight |
| `0 0 0 1 * *` | First day of every month |
| `0 0 0 L * *` | Last day of every month |
| `0 0 22 * * MON-FRI` | Weekdays at 10:00 PM |
| `0 */30 * * * *` | Every 30 minutes |
| `0 0 9-17 * * MON-FRI` | Every hour 9AM-5PM weekdays |
| `0 0 0 1 1,4,7,10 *` | First day of each quarter |

## 5.4 Externalized Cron in Production

```
// application.yml
scheduler:
  reconciliation-cron: "0 0 23 * * MON-FRI"
  cleanup-cron: "0 0 3 * * *"
  report-cron: "0 0 6 * * *"

// Scheduler class
@Scheduled(cron = "${scheduler.reconciliation-cron}")
public void runReconciliation() { ... }

@Scheduled(cron = "${scheduler.cleanup-cron}")
public void runCleanup() { ... }
```

---

# Part 6: Spring Boot Batch Processing

## 6.1 What is Spring Batch?

**Spring Batch** is a lightweight, comprehensive framework for developing robust batch applications. It provides reusable functions for processing large volumes of records, including:

- Reading from databases, files, queues
- Processing/transforming data
- Writing to databases, files, services
- Transaction management
- Job restart and retry
- Chunk-based processing
- Parallel execution

### Maven Dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

## 6.2 Why Spring Batch?

| Challenge | Spring Batch Solution |
|---|---|
| Processing millions of records | Chunk-based processing (configurable chunk size) |
| Transaction management | Automatic transaction per chunk |
| Failure recovery | Built-in restart, skip, retry mechanisms |
| Job monitoring | JobRepository tracks execution history |
| Scalability | Partitioning, multi-threaded steps, remote chunking |

| | |
|---|---|
| Reusability | Pre-built readers/writers for files, DB, JMS |

## 6.3 Real-World Scenarios

### Processing Millions of Records

```
Scenario: Insurance Company Premium Recalculation
- 2 million active policies
- Each policy needs:
  1. Risk factor recalculation
  2. Premium amount calculation
  3. Discount application
  4. Notice generation
- Must complete in 4-hour batch window (2 AM - 6 AM)
- Solution: Spring Batch with chunk size 500, 8 partitions
  -> 2,000,000 / 500 = 4,000 chunks
  -> 4,000 / 8 partitions = 500 chunks per partition
  -> ~30 minutes total processing time
```

### Daily Benefit Eligibility

```
Scenario: Government Benefits Portal
- 500,000 citizens enrolled
- Daily recalculation based on:
  1. Income changes
  2. Family size changes
  3. Employment status
  4. Age thresholds
- Generate eligibility notices for status changes
- Solution: Spring Batch reads citizens, processes eligibility rules,
  writes updated records and generates notices
```

### Bank Settlement Processing

```
Scenario: End-of-Day Settlement
- Process all inter-bank transactions
- Net settlement calculations
- Generate SWIFT messages
- Update general ledger
```

```
- Regulatory reporting
- Must complete before next business day opening
```

---

# Part 7: Spring Batch Architecture

## 7.1 Core Architecture

```
+-------------------------------------------------------------+
|                         JobLauncher                          |
|                    (Triggers job execution)                  |
+-------------------------------------------------------------+
         |                                        |
         v                                        v
+------------------+                    +------------------+
|       Job        |                    |  JobRepository   |
| (Batch workflow) |                    | (Execution       |
|                  |                    |  metadata store)  |
+------------------+                    +------------------+
         |
         v
+------------------+
|      Step 1      |  (Each step = one unit of work)
+------------------+
         |
         v
+------------------+     +------------------+     +------------------+
|   ItemReader     | --> |  ItemProcessor   | --> |   ItemWriter     |
| (Read data)      |     | (Transform data) |     | (Write data)     |
+------------------+     +------------------+     +------------------+
         |
         v
+------------------+
|      Step 2      |  (Jobs can have multiple steps)
+------------------+
         |
         v
+------------------+     +------------------+     +------------------+
|   ItemReader     | --> |  ItemProcessor   | --> |   ItemWriter     |
+------------------+     +------------------+     +------------------+
```

## 7.2 Chunk-Oriented Processing

Spring Batch processes data in **chunks**. A chunk is a group of items read, processed, and written together within a single transaction.

```
Chunk Processing Flow (chunk-size = 100):

1. READ Phase:
   Reader reads 100 items one by one
   [item1, item2, item3, ... item100]

2. PROCESS Phase:
   Processor transforms each item
   [processed1, processed2, ... processed100]

3. WRITE Phase:
   Writer writes all 100 items in one batch
   [write all 100 to database in single transaction]

4. COMMIT Transaction

5. Repeat for next chunk until no more items
```

### Why Chunk Processing?

- **Memory efficient** -- only one chunk in memory at a time
- **Transaction boundary** -- each chunk is one transaction
- **Failure recovery** -- restart from the last successful chunk
- **Performance** -- batch writes are faster than individual writes

---

# Part 8: Spring Batch Components

## 8.1 Job

**What:** The top-level container for a batch process. A Job is composed of one or more Steps.

**Internal:** `Job` interface is implemented by `SimpleJob` (sequential steps) or `FlowJob` (conditional flow).

```
@Bean
public Job invoiceGenerationJob(JobRepository jobRepository,
                                Step calculateStep,
                                Step generatePdfStep,
                                Step sendEmailStep) {
    return new JobBuilder("invoiceGenerationJob", jobRepository)
        .start(calculateStep)
        .next(generatePdfStep)
        .next(sendEmailStep)
        .build();
}
```

## 8.2 Step

**What:** A single phase of a Job. Each Step has exactly one `ItemReader`, `ItemProcessor`, and `ItemWriter`.

**Types:**

- **Chunk-oriented Step** -- reads, processes, writes in chunks (most common)
- **Tasklet Step** -- executes a single task (cleanup, file deletion, etc.)

```
@Bean
public Step calculateChargesStep(JobRepository jobRepository,
                                 PlatformTransactionManager txManager) {
    return new StepBuilder("calculateChargesStep", jobRepository)
        .<Customer, Invoice>chunk(100, txManager)  // Chunk size = 100
        .reader(customerReader())
        .processor(invoiceProcessor())
        .writer(invoiceWriter())
```

```
        .build();
}

// Tasklet Step (for simple tasks)
@Bean
public Step cleanupStep(JobRepository jobRepository,
                        PlatformTransactionManager txManager) {
    return new StepBuilder("cleanupStep", jobRepository)
        .tasklet((contribution, chunkContext) -> {
            fileService.deleteTemporaryFiles();
            return RepeatStatus.FINISHED;
        }, txManager)
        .build();
}
```

## 8.3 ItemReader

**What:** Reads data from a source (database, file, queue, API). Returns one item at a time; returns `null` when no more items.

**Built-in Readers:**

| Reader | Source | Use Case |
|---|---|---|
| `JdbcCursorItemReader` | Database (JDBC cursor) | Small-medium datasets |
| `JdbcPagingItemReader` | Database (paging queries) | Large datasets |
| `JpaPagingItemReader` | Database (JPA paging) | JPA-based applications |
| `FlatFileItemReader` | CSV/text files | File processing |
| `JsonItemReader` | JSON files | JSON data import |
| `StaxEventItemReader` | XML files | XML data import |

```
@Bean
public JpaPagingItemReader<Customer> customerReader(EntityManagerFactory emf) {
    return new JpaPagingItemReaderBuilder<Customer>()
        .name("customerReader")
        .entityManagerFactory(emf)
        .queryString("SELECT c FROM Customer c WHERE c.active = true")
        .pageSize(100)
```

```
        .build();
}
```

## 8.4 ItemProcessor

**What:** Transforms or filters items. Takes one item, returns a transformed item (or `null` to skip).

```
@Component
public class InvoiceProcessor implements ItemProcessor<Customer, Invoice> {

    @Autowired
    private PricingService pricingService;

    @Override
    public Invoice process(Customer customer) throws Exception {
        // Skip inactive customers
        if (!customer.isActive()) {
            return null; // Returning null = skip this item
        }

        Invoice invoice = new Invoice();
        invoice.setCustomerId(customer.getId());
        invoice.setAmount(pricingService.calculate(customer));
        invoice.setDueDate(LocalDate.now().plusDays(30));
        invoice.setStatus(InvoiceStatus.GENERATED);
        return invoice;
    }
}
```

## 8.5 ItemWriter

**What:** Writes a list of items (one chunk) to the target (database, file, API).

```
@Bean
public JpaItemWriter<Invoice> invoiceWriter(EntityManagerFactory emf) {
    JpaItemWriter<Invoice> writer = new JpaItemWriter<>();
    writer.setEntityManagerFactory(emf);
    return writer;
}

// Custom writer for complex scenarios
```

```java
@Component
public class NoticeWriter implements ItemWriter<Notice> {

    @Autowired
    private NoticeRepository noticeRepository;
    @Autowired
    private PdfService pdfService;

    @Override
    public void write(Chunk<? extends Notice> notices) throws Exception {
        for (Notice notice : notices) {
            byte[] pdf = pdfService.generate(notice);
            notice.setPdfContent(pdf);
            notice.setGeneratedAt(Instant.now());
        }
        noticeRepository.saveAll(notices.getItems());
    }
}
```

## 8.6 JobRepository

**What:** Stores metadata about job executions (status, parameters, start/end times). Uses database tables (BATCH_JOB_INSTANCE, BATCH_JOB_EXECUTION, etc.).

## 8.7 JobLauncher

**What:** Starts a Job with given parameters. Can run synchronously or asynchronously.

```java
@Autowired
private JobLauncher jobLauncher;

@Autowired
private Job invoiceJob;

public void triggerJob() throws Exception {
    JobParameters params = new JobParametersBuilder()
        .addString("date", LocalDate.now().toString())
        .addLong("timestamp", System.currentTimeMillis())
        .toJobParameters();

    JobExecution execution = jobLauncher.run(invoiceJob, params);
```

```
    log.info("Job Status: {}", execution.getStatus());
}
```

## 8.8 ExecutionContext

**What:** Key-value store for sharing data between steps or across restarts.

```
// In Step 1: Store data
chunkContext.getStepContext().getStepExecution()
    .getJobExecution().getExecutionContext()
    .put("totalProcessed", count);

// In Step 2: Retrieve data
int totalProcessed = chunkContext.getStepContext()
    .getStepExecution().getJobExecution()
    .getExecutionContext().getInt("totalProcessed");
```

---

# Part 9: Spring Batch Project Structure

## 9.1 Recommended Production Structure

```
src/main/java/com/company/batchservice/
|
+-- config/
|   +-- BatchConfig.java              // Job and Step bean definitions
|   +-- DataSourceConfig.java         // Database configuration
|   +-- SchedulerConfig.java          // Scheduler thread pool config
|
+-- job/
|   +-- InvoiceGenerationJobConfig.java    // Job definition
|   +-- EligibilityJobConfig.java
|   +-- ReconciliationJobConfig.java
|
+-- reader/
|   +-- CustomerItemReader.java
|   +-- TransactionItemReader.java
|   +-- CsvFileItemReader.java
|
+-- processor/
|   +-- InvoiceProcessor.java
|   +-- EligibilityProcessor.java
|   +-- DataTransformProcessor.java
|
+-- writer/
|   +-- InvoiceItemWriter.java
|   +-- NoticeItemWriter.java
|   +-- CsvFileItemWriter.java
|
+-- scheduler/
|   +-- BatchJobScheduler.java         // Cron triggers for jobs
|
+-- listener/
|   +-- JobCompletionListener.java     // Job lifecycle callbacks
|   +-- StepExecutionListener.java
|
+-- entity/
|   +-- Customer.java
|   +-- Invoice.java
|   +-- Notice.java
|
```

```
+-- repository/
|   +-- CustomerRepository.java
|   +-- InvoiceRepository.java
|
+-- service/
|   +-- PricingService.java
|   +-- PdfService.java
|   +-- EmailService.java
|
+-- controller/
|   +-- BatchJobController.java        // REST endpoints to trigger jobs
|
+-- exception/
    +-- BatchProcessingException.java
```

## 9.2 Purpose of Each Package

| Package | Purpose |
|---|---|
| `config/` | Spring configuration classes, data sources, thread pools |
| `job/` | Job definitions with steps, flows, and conditional logic |
| `reader/` | Custom ItemReader implementations |
| `processor/` | Business logic for data transformation |
| `writer/` | Custom ItemWriter implementations |
| `scheduler/` | Cron-based job triggers |
| `listener/` | Job and Step lifecycle callbacks (logging, alerts) |
| `entity/` | JPA entity classes |
| `repository/` | Spring Data JPA repositories |
| `service/` | Shared business services |
| `controller/` | REST APIs for manual job triggers |

---

# Part 10: Complete Batch Job Implementation

## 10.1 Entity

```java
@Entity
@Table(name = "customers")
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String plan; // BASIC, PREMIUM, ENTERPRISE
    private BigDecimal usageAmount;
    private boolean active;
    private LocalDate billingCycleStart;

    // Getters and setters
}

@Entity
@Table(name = "invoices")
public class Invoice {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long customerId;
    private String customerName;
    private String customerEmail;
    private BigDecimal amount;
    private BigDecimal tax;
    private BigDecimal totalAmount;
    private LocalDate invoiceDate;
    private LocalDate dueDate;
    private String status; // GENERATED, SENT, PAID

    // Getters and setters
}
```

## 10.2 Reader

```java
@Configuration
public class CustomerReaderConfig {

    @Bean
    public JpaPagingItemReader<Customer> activeCustomerReader(
            EntityManagerFactory entityManagerFactory) {
        return new JpaPagingItemReaderBuilder<Customer>()
            .name("activeCustomerReader")
            .entityManagerFactory(entityManagerFactory)
            .queryString("SELECT c FROM Customer c WHERE c.active = true " +
                         "ORDER BY c.id")
            .pageSize(200)
            .build();
    }
}
```

## 10.3 Processor

```java
@Component
public class InvoiceGenerationProcessor
        implements ItemProcessor<Customer, Invoice> {

    private static final Map<String, BigDecimal> PLAN_RATES = Map.of(
        "BASIC", new BigDecimal("29.99"),
        "PREMIUM", new BigDecimal("79.99"),
        "ENTERPRISE", new BigDecimal("199.99")
    );
    private static final BigDecimal TAX_RATE = new BigDecimal("0.18");

    @Override
    public Invoice process(Customer customer) throws Exception {
        if (!customer.isActive()) {
            return null; // Skip inactive
        }

        BigDecimal baseAmount = PLAN_RATES.getOrDefault(
            customer.getPlan(), BigDecimal.ZERO);
        BigDecimal usage = customer.getUsageAmount() != null
            ? customer.getUsageAmount() : BigDecimal.ZERO;
        BigDecimal subtotal = baseAmount.add(usage);
        BigDecimal tax = subtotal.multiply(TAX_RATE)
            .setScale(2, RoundingMode.HALF_UP);
```

```
        BigDecimal total = subtotal.add(tax);

        Invoice invoice = new Invoice();
        invoice.setCustomerId(customer.getId());
        invoice.setCustomerName(customer.getName());
        invoice.setCustomerEmail(customer.getEmail());
        invoice.setAmount(subtotal);
        invoice.setTax(tax);
        invoice.setTotalAmount(total);
        invoice.setInvoiceDate(LocalDate.now());
        invoice.setDueDate(LocalDate.now().plusDays(30));
        invoice.setStatus("GENERATED");
        return invoice;
    }
}
```

## 10.4 Writer

```
@Component
public class InvoiceItemWriter implements ItemWriter<Invoice> {

    @Autowired
    private InvoiceRepository invoiceRepository;

    @Override
    public void write(Chunk<? extends Invoice> invoices) throws Exception {
        invoiceRepository.saveAll(invoices.getItems());
        log.info("Written {} invoices to database", invoices.size());
    }
}
```

## 10.5 Job Configuration

```
@Configuration
public class InvoiceJobConfig {

    @Bean
    public Job invoiceGenerationJob(JobRepository jobRepository,
                                    Step invoiceStep,
                                    JobCompletionListener listener) {
```

```
        return new JobBuilder("invoiceGenerationJob", jobRepository)
            .listener(listener)
            .start(invoiceStep)
            .build();
    }

    @Bean
    public Step invoiceStep(JobRepository jobRepository,
                            PlatformTransactionManager transactionManager,
                            JpaPagingItemReader<Customer> activeCustomerReader,
                            InvoiceGenerationProcessor processor,
                            InvoiceItemWriter writer) {
        return new StepBuilder("invoiceStep", jobRepository)
            .<Customer, Invoice>chunk(100, transactionManager)
            .reader(activeCustomerReader)
            .processor(processor)
            .writer(writer)
            .faultTolerant()
            .skipLimit(10)
            .skip(Exception.class)
            .retryLimit(3)
            .retry(TransientDataAccessException.class)
            .build();
    }
}
```

## 10.6 Job Completion Listener

```
@Component
public class JobCompletionListener implements JobExecutionListener {

    @Autowired
    private EmailService emailService;

    @Override
    public void beforeJob(JobExecution jobExecution) {
        log.info("Job {} STARTED at {}",
            jobExecution.getJobInstance().getJobName(),
            jobExecution.getStartTime());
    }

    @Override
    public void afterJob(JobExecution jobExecution) {
        BatchStatus status = jobExecution.getStatus();
```

```
        String jobName = jobExecution.getJobInstance().getJobName();

        if (status == BatchStatus.COMPLETED) {
            log.info("Job {} COMPLETED successfully", jobName);
            emailService.sendJobReport(jobName, "SUCCESS",
                jobExecution.getStepExecutions());
        } else if (status == BatchStatus.FAILED) {
            log.error("Job {} FAILED", jobName);
            emailService.sendJobAlert(jobName, "FAILURE",
                jobExecution.getAllFailureExceptions());
        }
    }
}
```

---

# Part 11: Real-Time Production Scenario

## 11.1 Daily Client Notice Generation System

### Business Requirement

A financial services company needs to:

1. Every night at midnight, check all client accounts
2. Identify clients whose eligibility status changed
3. Generate PDF notices for each affected client
4. Store the PDF in the document management system
5. Send email notifications to clients

### Architecture

```
+-----------------------------------------------------------------------+
|                     Notice Generation System                          |
|                                                                       |
|  [Scheduler]                                                          |
|  cron: 0 0 0 * * *   --trigger-->                                    |
|                                                                       |
|  +------------------------------- JOB ----------------------------+   |
|  |                                                                 |   |
|  |  STEP 1: Eligibility Check                                     |   |
|  |  +----------+    +-----------+    +----------+                |   |
|  |  | Client   | -> | Eligib.   | -> | Status   |                |   |
|  |  | Reader   |    | Processor |    | Writer   |                |   |
|  |  |(Database)|    |(Rules Eng)|    |(Database)|                |   |
|  |  +----------+    +-----------+    +----------+                |   |
|  |                                                                 |   |
|  |  STEP 2: Notice Generation                                     |   |
|  |  +----------+    +-----------+    +----------+                |   |
|  |  | Changed  | -> | PDF       | -> | Notice   |                |   |
|  |  | Clients  |    | Generator |    | Writer   |                |   |
|  |  | Reader   |    | Processor |    |(DB+Email)|                |   |
|  |  +----------+    +-----------+    +----------+                |   |
|  |                                                                 |   |
|  +-----------------------------------------------------------------+   |
+-----------------------------------------------------------------------+
```

### Implementation

```java
// Step 1: Check eligibility changes
@Bean
public Step eligibilityCheckStep(JobRepository jobRepository,
        PlatformTransactionManager txManager) {
    return new StepBuilder("eligibilityCheckStep", jobRepository)
        .<Client, Client>chunk(200, txManager)
        .reader(allActiveClientsReader())
        .processor(eligibilityCheckProcessor())
        .writer(statusUpdateWriter())
        .build();
}

// Step 2: Generate notices for changed statuses
@Bean
public Step noticeGenerationStep(JobRepository jobRepository,
        PlatformTransactionManager txManager) {
    return new StepBuilder("noticeGenerationStep", jobRepository)
        .<Client, Notice>chunk(50, txManager)
        .reader(changedStatusClientsReader())
        .processor(noticeGenerationProcessor())
        .writer(noticeWriter())
        .build();
}

// Job: Wire both steps
@Bean
public Job dailyNoticeJob(JobRepository jobRepository,
                          Step eligibilityCheckStep,
                          Step noticeGenerationStep) {
    return new JobBuilder("dailyNoticeJob", jobRepository)
        .start(eligibilityCheckStep)
        .next(noticeGenerationStep)
        .build();
}

// Processor: Generate PDF notice
@Component
public class NoticeGenerationProcessor
        implements ItemProcessor<Client, Notice> {

    @Autowired
    private PdfService pdfService;

    @Override
```

```java
    public Notice process(Client client) throws Exception {
        Notice notice = new Notice();
        notice.setClientId(client.getId());
        notice.setClientEmail(client.getEmail());
        notice.setType(determineNoticeType(client));
        notice.setContent(buildNoticeContent(client));
        notice.setPdfBytes(pdfService.generateNoticePdf(notice));
        notice.setGeneratedAt(Instant.now());
        notice.setStatus("GENERATED");
        return notice;
    }
}

// Writer: Save notice + Send email
@Component
public class NoticeAndEmailWriter implements ItemWriter<Notice> {

    @Autowired private NoticeRepository noticeRepository;
    @Autowired private EmailService emailService;

    @Override
    public void write(Chunk<? extends Notice> notices) throws Exception {
        List<Notice> saved = noticeRepository.saveAll(notices.getItems());
        for (Notice notice : saved) {
            try {
                emailService.sendNoticeEmail(
                    notice.getClientEmail(),
                    notice.getType(),
                    notice.getPdfBytes()
                );
                notice.setStatus("SENT");
            } catch (Exception e) {
                notice.setStatus("EMAIL_FAILED");
                log.error("Email failed for client {}", notice.getClientId(),
e);
            }
        }
        noticeRepository.saveAll(saved);
    }
}
```

# Part 12: Handling Large Datasets

## 12.1 Chunk Processing

The fundamental strategy. Configure chunk size based on dataset and memory:

```
// Small dataset (< 10K records): larger chunks
.<Entity, Entity>chunk(500, txManager)

// Medium dataset (10K - 1M): moderate chunks
.<Entity, Entity>chunk(100, txManager)

// Large dataset (> 1M): smaller chunks
.<Entity, Entity>chunk(50, txManager)
```

| Chunk Size | Memory Usage | Transaction Size | Restart Granularity |
|---|---|---|---|
| 10 | Very Low | Small | Fine-grained |
| 100 | Low | Medium | Balanced |
| 500 | Medium | Large | Coarse |
| 1000 | High | Very Large | Very Coarse |

## 12.2 Paging vs Cursor Reading

### Paging Reader (Recommended for Large Datasets)

```
@Bean
public JpaPagingItemReader<Transaction> transactionReader(
        EntityManagerFactory emf) {
    return new JpaPagingItemReaderBuilder<Transaction>()
        .name("transactionReader")
        .entityManagerFactory(emf)
        .queryString("SELECT t FROM Transaction t WHERE t.status = 'PENDING'")
        .pageSize(500) // Fetches 500 records per DB query
        .build();
}
// How it works:
```

```
// Page 0: SELECT ... LIMIT 500 OFFSET 0
// Page 1: SELECT ... LIMIT 500 OFFSET 500
// Page 2: SELECT ... LIMIT 500 OFFSET 1000
// ...until no more results
```

### Cursor Reader (For Medium Datasets)

```java
@Bean
public JdbcCursorItemReader<Transaction> cursorReader(DataSource ds) {
    return new JdbcCursorItemReaderBuilder<Transaction>()
        .name("cursorReader")
        .dataSource(ds)
        .sql("SELECT * FROM transactions WHERE status = 'PENDING'")
        .rowMapper(new TransactionRowMapper())
        .fetchSize(100) // JDBC fetch size
        .build();
}
// Opens a DB cursor, streams results row by row
// Lower memory but holds DB connection longer
```

## 12.3 Multi-Threaded Step

```java
@Bean
public Step multiThreadedStep(JobRepository jobRepo,
                              PlatformTransactionManager txManager) {
    return new StepBuilder("multiThreadedStep", jobRepo)
        .<Customer, Invoice>chunk(100, txManager)
        .reader(customerReader())   // Must be thread-safe!
        .processor(invoiceProcessor())
        .writer(invoiceWriter())
        .taskExecutor(new SimpleAsyncTaskExecutor())
        .throttleLimit(8) // 8 concurrent threads
        .build();
}
```

## 12.4 Partitioning

Split data into independent partitions, each processed by a separate thread:

```java
@Bean
public Step partitionedStep(JobRepository jobRepo,
                            Step workerStep) {
    return new StepBuilder("partitionedStep", jobRepo)
        .partitioner("workerStep", rangePartitioner())
        .step(workerStep)
        .gridSize(8)  // 8 partitions
        .taskExecutor(new SimpleAsyncTaskExecutor())
        .build();
}

@Bean
public Partitioner rangePartitioner() {
    return gridSize -> {
        Map<String, ExecutionContext> partitions = new HashMap<>();
        long min = customerRepository.findMinId();
        long max = customerRepository.findMaxId();
        long range = (max - min) / gridSize + 1;

        for (int i = 0; i < gridSize; i++) {
            ExecutionContext context = new ExecutionContext();
            context.putLong("minId", min + (i * range));
            context.putLong("maxId", min + ((i + 1) * range) - 1);
            partitions.put("partition" + i, context);
        }
        return partitions;
    };
}
```

# Part 13: Error Handling in Batch Jobs

## 13.1 Skip Logic

Skip problematic records and continue processing. Essential for batch jobs where a few bad records should not fail the entire job.

```
@Bean
public Step faultTolerantStep(JobRepository jobRepo,
                              PlatformTransactionManager txManager) {
    return new StepBuilder("faultTolerantStep", jobRepo)
        .<Customer, Invoice>chunk(100, txManager)
        .reader(customerReader())
        .processor(invoiceProcessor())
        .writer(invoiceWriter())
        .faultTolerant()
        .skipLimit(50)                              // Max 50 skips allowed
        .skip(DataIntegrityViolationException.class) // Skip on this exception
        .skip(InvalidDataException.class)
        .noSkip(DatabaseConnectionException.class)   // Never skip on this
        .listener(skipListener())                    // Log skipped items
        .build();
}

@Component
public class BatchSkipListener implements SkipListener<Customer, Invoice> {

    @Override
    public void onSkipInRead(Throwable t) {
        log.error("Skipped during READ: {}", t.getMessage());
    }

    @Override
    public void onSkipInProcess(Customer customer, Throwable t) {
        log.error("Skipped during PROCESS for customer {}: {}",
            customer.getId(), t.getMessage());
        auditService.logSkippedRecord("PROCESS", customer.getId(), t);
    }

    @Override
    public void onSkipInWrite(Invoice invoice, Throwable t) {
        log.error("Skipped during WRITE for invoice {}: {}",
            invoice.getId(), t.getMessage());
```

```
    }
}
```

## 13.2 Retry Logic

Automatically retry failed operations (useful for transient errors like DB timeouts):

```
@Bean
public Step retryableStep(JobRepository jobRepo,
                          PlatformTransactionManager txManager) {
    return new StepBuilder("retryableStep", jobRepo)
        .<Order, Settlement>chunk(100, txManager)
        .reader(orderReader())
        .processor(settlementProcessor())
        .writer(settlementWriter())
        .faultTolerant()
        .retryLimit(3)                                  // Retry up to 3 times
        .retry(TransientDataAccessException.class)      // Retry on DB timeout
        .retry(OptimisticLockingFailureException.class)
        .noRetry(ValidationException.class)             // Don't retry
validation errors
        .build();
}
```

## 13.3 Custom Retry and Skip Together

```
@Bean
public Step robustStep(JobRepository jobRepo,
                       PlatformTransactionManager txManager) {
    return new StepBuilder("robustStep", jobRepo)
        .<Input, Output>chunk(100, txManager)
        .reader(reader())
        .processor(processor())
        .writer(writer())
        .faultTolerant()
        // Retry transient errors 3 times
        .retryLimit(3)
        .retry(DeadlockLoserDataAccessException.class)
        .retry(CannotAcquireLockException.class)
        // After retries exhausted, skip and continue
```

```
        .skipLimit(100)
        .skip(Exception.class)
        .noSkip(FatalBatchException.class)  // Never skip fatal errors
        .listener(skipListener())
        .listener(retryListener())
        .build();
}
```

## 13.4 Transaction Rollback

By default, Spring Batch rolls back the entire chunk transaction on error. You can configure exceptions that should NOT cause rollback:

```
.noRollback(ValidationException.class)  // Don't rollback for validation errors
```

---

# Part 14: Job Monitoring

## 14.1 JobExecution

```
@RestController
@RequestMapping("/api/batch")
public class BatchMonitorController {

    @Autowired
    private JobExplorer jobExplorer;

    @GetMapping("/jobs/{jobName}/executions")
    public List<JobExecutionDTO> getExecutions(@PathVariable String jobName) {
        List<JobInstance> instances = jobExplorer
            .getJobInstances(jobName, 0, 20);

        return instances.stream()
            .flatMap(instance ->
                jobExplorer.getJobExecutions(instance).stream())
            .map(exec -> new JobExecutionDTO(
                exec.getId(),
                exec.getStatus().toString(),
                exec.getStartTime(),
                exec.getEndTime(),
                exec.getExitStatus().getExitCode(),
                getStepSummaries(exec)
            ))
            .collect(Collectors.toList());
    }
}
```

## 14.2 StepExecution Monitoring

```
@Component
public class StepMonitorListener implements StepExecutionListener {

    @Override
    public void afterStep(StepExecution stepExecution) {
        log.info("Step: {} | Status: {} | Read: {} | Processed: {} | " +
```

```
                "Written: {} | Skipped: {} | Duration: {}ms",
            stepExecution.getStepName(),
            stepExecution.getStatus(),
            stepExecution.getReadCount(),
            stepExecution.getFilterCount(),
            stepExecution.getWriteCount(),
            stepExecution.getSkipCount(),
            Duration.between(stepExecution.getStartTime(),
                stepExecution.getEndTime()).toMillis()
        );

        // Send metrics to Prometheus/Grafana
        metricsService.recordStepMetrics(
            stepExecution.getStepName(),
            stepExecution.getReadCount(),
            stepExecution.getWriteCount(),
            stepExecution.getSkipCount()
        );
    }
}
```

## 14.3 Production Monitoring Dashboard Metrics

| Metric | What to Monitor |
|---|---|
| `batch.job.duration` | Total job execution time |
| `batch.step.read.count` | Records read per step |
| `batch.step.write.count` | Records written per step |
| `batch.step.skip.count` | Records skipped (errors) |
| `batch.job.status` | COMPLETED, FAILED, STOPPED |
| `batch.step.throughput` | Records processed per second |
| Queue depth | Unprocessed items in work queue |
| Thread pool utilization | Active threads vs pool size |

---

# Part 15: Spring Batch Database Tables

## 15.1 Internal Tables

Spring Batch automatically creates metadata tables to track job execution:

| Table | Purpose |
|---|---|
| `BATCH_JOB_INSTANCE` | Logical job run (unique per job name + parameters) |
| `BATCH_JOB_EXECUTION` | Physical job run (each attempt, including restarts) |
| `BATCH_JOB_EXECUTION_PARAMS` | Parameters passed to the job |
| `BATCH_JOB_EXECUTION_CONTEXT` | Serialized ExecutionContext for the job |
| `BATCH_STEP_EXECUTION` | Each step's execution details |
| `BATCH_STEP_EXECUTION_CONTEXT` | Serialized ExecutionContext for the step |

## 15.2 Table Relationships

```
BATCH_JOB_INSTANCE (1) -----> (*) BATCH_JOB_EXECUTION
     |                                    |
     | (job name + params = unique)       | (each run/restart)
     |                                    |
     |                                    +-----> (*) BATCH_STEP_EXECUTION
     |                                    |           (one per step per
execution)
     |                                    |
     |                                    +-----> (*)
BATCH_JOB_EXECUTION_PARAMS
     |                                    |
     |                                    +-----> (1)
BATCH_JOB_EXECUTION_CONTEXT
```

## 15.3 Key Fields

### BATCH_JOB_EXECUTION

| Column | Description |
|---|---|
| `JOB_EXECUTION_ID` | Primary key |
| `JOB_INSTANCE_ID` | FK to job instance |
| `START_TIME` | When the job started |
| `END_TIME` | When the job ended |
| `STATUS` | COMPLETED, STARTED, FAILED, STOPPED, ABANDONED |
| `EXIT_CODE` | COMPLETED, FAILED, NOOP, etc. |
| `EXIT_MESSAGE` | Error message if failed |

### BATCH_STEP_EXECUTION

| Column | Description |
|---|---|
| `STEP_EXECUTION_ID` | Primary key |
| `STEP_NAME` | Name of the step |
| `READ_COUNT` | Number of items read |
| `WRITE_COUNT` | Number of items written |
| `COMMIT_COUNT` | Number of chunks committed |
| `SKIP_COUNT_READ/PROCESS/WRITE` | Skipped items at each phase |
| `ROLLBACK_COUNT` | Number of rollbacks |

## 15.4 Configuration

```yaml
# application.yml
spring:
  batch:
    jdbc:
      initialize-schema: always  # Create tables on startup (dev)
      # initialize-schema: never  # Production (use Flyway/Liquibase)
    job:
      enabled: false  # Don't auto-run jobs on startup
```

---

# Part 16: Spring Batch + Scheduler Integration

## 16.1 Trigger Batch Jobs with Scheduler

```
@Component
public class BatchJobScheduler {

    @Autowired
    private JobLauncher jobLauncher;

    @Autowired
    @Qualifier("invoiceGenerationJob")
    private Job invoiceJob;

    @Autowired
    @Qualifier("dailyNoticeJob")
    private Job noticeJob;

    // Trigger invoice generation at midnight on the 1st of each month
    @Scheduled(cron = "0 0 0 1 * *")
    public void triggerMonthlyInvoiceJob() {
        try {
            JobParameters params = new JobParametersBuilder()
                .addString("month", YearMonth.now().toString())
                .addLong("timestamp", System.currentTimeMillis())
                .toJobParameters();

            JobExecution execution = jobLauncher.run(invoiceJob, params);
            log.info("Invoice job started with status: {}",
                execution.getStatus());
        } catch (Exception e) {
            log.error("Failed to start invoice job", e);
            alertService.sendCriticalAlert("Invoice batch job failed to start",
e);
        }
    }

    // Trigger notice generation every midnight
    @Scheduled(cron = "0 0 0 * * *")
    public void triggerDailyNoticeJob() {
        try {
            JobParameters params = new JobParametersBuilder()
                .addString("date", LocalDate.now().toString())
```

```
                .addLong("timestamp", System.currentTimeMillis())
                .toJobParameters();

            jobLauncher.run(noticeJob, params);
        } catch (Exception e) {
            log.error("Failed to start notice job", e);
        }
    }
}
```

## 16.2 REST API Trigger (Manual Execution)

```
@RestController
@RequestMapping("/api/batch")
public class BatchJobController {

    @Autowired private JobLauncher jobLauncher;
    @Autowired private Job invoiceJob;

    @PostMapping("/jobs/invoice/run")
    public ResponseEntity<String> runInvoiceJob(
            @RequestParam String month) {
        try {
            JobParameters params = new JobParametersBuilder()
                .addString("month", month)
                .addLong("timestamp", System.currentTimeMillis())
                .toJobParameters();

            JobExecution execution = jobLauncher.run(invoiceJob, params);
            return ResponseEntity.ok(
                "Job started with ID: " + execution.getId());
        } catch (JobInstanceAlreadyCompleteException e) {
            return ResponseEntity.status(409)
                .body("Job already completed for this month");
        } catch (Exception e) {
            return ResponseEntity.status(500)
                .body("Failed to start job: " + e.getMessage());
        }
    }
}
```

---

# Part 17: Microservice Architecture with Batch

## 17.1 Architecture Overview

```
+-----------------------------------------------------------------+
|                      API Gateway                                |
+-----------------------------------------------------------------+
     |              |                |                |
     v              v                v                v
+----------+  +----------+  +--------------+  +--------------+
| Eligib.  |  | Notice   |  | Notification |  | Batch        |
| Service  |  | Service  |  | Service      |  | Processor    |
| (REST)   |  | (REST)   |  | (REST)       |  | Service      |
+----------+  +----------+  +--------------+  +--------------+
     |              |                |                |
     v              v                v                v
+----------+  +----------+  +--------------+  +--------------+
| Eligib.  |  | Notice   |  | Email/SMS    |  | Batch Jobs   |
| Rules    |  | Template |  | Gateway      |  | Spring Batch |
| Engine   |  | Engine   |  |              |  | + Scheduler  |
+----------+  +----------+  +--------------+  +--------------+
     |              |                                |
     +-------+------+                                |
             |                                       |
             v                                       v
     +--------------+                        +--------------+
     |  PostgreSQL  |                        |  PostgreSQL  |
     |  (Services)  |                        |  (Batch Meta)|
     +--------------+                        +--------------+
             |
             v
     +--------------+
     |    Kafka     |  (Event-driven communication)
     +--------------+
```

## 17.2 Batch Processing Service

```
@SpringBootApplication
@EnableBatchProcessing
```

```java
@EnableScheduling
public class BatchProcessorServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(BatchProcessorServiceApplication.class, args);
    }
}
```

### Inter-Service Communication from Batch

```java
@Component
public class EligibilityCheckProcessor
        implements ItemProcessor<Client, Client> {

    @Autowired
    private WebClient eligibilityServiceClient;

    @Override
    public Client process(Client client) throws Exception {
        // Call Eligibility Service via REST
        EligibilityResult result = eligibilityServiceClient.get()
            .uri("/api/eligibility/{clientId}", client.getId())
            .retrieve()
            .bodyToMono(EligibilityResult.class)
            .block(Duration.ofSeconds(5));

        if (result != null && result.isStatusChanged()) {
            client.setEligibilityStatus(result.getNewStatus());
            return client;
        }
        return null; // Skip unchanged
    }
}

// Writer: Publish event to Kafka for Notification Service
@Component
public class NotificationEventWriter implements ItemWriter<Notice> {

    @Autowired
    private KafkaTemplate<String, NoticeEvent> kafkaTemplate;

    @Override
    public void write(Chunk<? extends Notice> notices) throws Exception {
        for (Notice notice : notices) {
            NoticeEvent event = new NoticeEvent(
                notice.getClientId(),
```

```
                notice.getType(),
                notice.getClientEmail()
            );
            kafkaTemplate.send("notice-events", event);
        }
    }
}
```

---

# Part 18: Production Best Practices

## 18.1 Idempotency

Every batch job must be **idempotent** -- running it twice with the same parameters should produce the same result without side effects.

```
// BAD: Not idempotent -- running twice creates duplicate invoices
public Invoice process(Customer customer) {
    return new Invoice(customer); // Always creates new
}

// GOOD: Idempotent -- checks for existing before creating
public Invoice process(Customer customer) {
    Invoice existing = invoiceRepository
        .findByCustomerIdAndMonth(customer.getId(), YearMonth.now());
    if (existing != null) {
        return null; // Skip -- already processed
    }
    return new Invoice(customer);
}
```

## 18.2 Restartable Jobs

Configure jobs to restart from the last failed point, not from the beginning:

```
@Bean
public Step restartableStep(JobRepository jobRepo,
                            PlatformTransactionManager txManager) {
    return new StepBuilder("restartableStep", jobRepo)
        .<Data, Data>chunk(100, txManager)
        .reader(pagingReader()) // Paging readers are restartable!
        .processor(processor())
        .writer(writer())
        .allowStartIfComplete(false) // Don't re-run completed steps
        .build();
}
```

## 18.3 Job Parameters for Uniqueness

```
// Use meaningful parameters so each run is unique
JobParameters params = new JobParametersBuilder()
    .addString("date", LocalDate.now().toString())     // Makes each day unique
    .addLong("timestamp", System.currentTimeMillis()) // Forces uniqueness
    .toJobParameters();
```

## 18.4 Externalize Configuration

```
# application.yml
batch:
  invoice-job:
    chunk-size: 100
    skip-limit: 50
    retry-limit: 3
    thread-count: 4
    cron: "0 0 0 1 * *"
```

```
@Value("${batch.invoice-job.chunk-size}")
private int chunkSize;

@Value("${batch.invoice-job.skip-limit}")
private int skipLimit;
```

## 18.5 Monitoring and Alerting

```
@Component
public class JobCompletionNotifier implements JobExecutionListener {

    @Override
    public void afterJob(JobExecution execution) {
        // Send metrics to monitoring system
        meterRegistry.timer("batch.job.duration",
            "job", execution.getJobInstance().getJobName(),
```

```
            "status", execution.getStatus().toString())
            .record(Duration.between(
                execution.getStartTime(), execution.getEndTime()));

        if (execution.getStatus() == BatchStatus.FAILED) {
            // Page the on-call engineer
            pagerDutyService.createIncident(
                "Batch Job Failed: " +
                execution.getJobInstance().getJobName(),
                execution.getAllFailureExceptions()
            );
        }
    }
}
```

# Part 19: Performance Optimization

## 19.1 Thread Pool Configuration

```
@Bean
public TaskExecutor batchTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(4);
    executor.setMaxPoolSize(8);
    executor.setQueueCapacity(50);
    executor.setThreadNamePrefix("batch-");
    executor.initialize();
    return executor;
}
```

## 19.2 Parallel Steps

Execute independent steps concurrently:

```
@Bean
public Job parallelJob(JobRepository jobRepo,
                       Step loadCustomers,
                       Step loadProducts,
                       Step generateReport) {
    // loadCustomers and loadProducts run in parallel
    // generateReport runs after both complete
    Flow customerFlow = new FlowBuilder<SimpleFlow>("customerFlow")
        .start(loadCustomers).build();
    Flow productFlow = new FlowBuilder<SimpleFlow>("productFlow")
        .start(loadProducts).build();

    return new JobBuilder("parallelJob", jobRepo)
        .start(new FlowBuilder<SimpleFlow>("parallelFlow")
            .split(new SimpleAsyncTaskExecutor())
            .add(customerFlow, productFlow)
            .build())
        .next(generateReport) // Runs after both complete
        .end()
```

```
        .build();
}
```

## 19.3 Optimization Checklist

| Optimization | Impact | How |
|---|---|---|
| Chunk size tuning | High | Start with 100, benchmark and adjust |
| Paging reader | High | Use pageSize matching chunk size |
| Multi-threaded steps | High | `taskExecutor()` + `throttleLimit()` |
| Partitioning | Very High | Split data into independent ranges |
| Index DB queries | Very High | Index columns used in reader queries |
| Batch writes | High | Use `JdbcBatchItemWriter` for bulk inserts |
| Disable auto-flush | Medium | Set `clearSession=true` on JPA reader |
| Connection pool | Medium | HikariCP with adequate max-pool-size |
| Parallel steps | Medium | Independent steps run concurrently |
| Async writer | Medium | Write to queue, process asynchronously |

## 19.4 Benchmarking Example

```
Scenario: Process 1 million customer records

Configuration A: Single-threaded, chunk=100
  -> Time: 45 minutes

Configuration B: 4 threads, chunk=100
  -> Time: 13 minutes

Configuration C: 8 partitions, chunk=200
  -> Time: 6 minutes

Configuration D: 8 partitions, chunk=200, indexed queries
  -> Time: 3 minutes
```

---

# Part 20: Common Production Issues

## 20.1 Job Failures

**Problem:** Job fails midway through processing 500,000 records.

**Solution:**

```
// 1. Make jobs restartable (default in Spring Batch)
// 2. Use paging readers (maintain position via page number)
// 3. Check job status and restart
JobExecution lastExecution = jobExplorer
    .getLastJobExecution(jobInstance);
if (lastExecution != null &&
    lastExecution.getStatus() == BatchStatus.FAILED) {
    // Restart from last committed chunk
    jobLauncher.run(job, lastExecution.getJobParameters());
}
```

## 20.2 Duplicate Processing

**Problem:** Job runs twice for the same date, creating duplicate records.

**Solution:**

```
// Option 1: Unique job parameters prevent re-execution
// Spring Batch won't re-run a COMPLETED job with same parameters

// Option 2: Idempotent processor
public Invoice process(Customer customer) {
    if (invoiceRepository.existsByCustomerIdAndMonth(
            customer.getId(), currentMonth)) {
        return null; // Already processed, skip
    }
    return createInvoice(customer);
}

// Option 3: Database unique constraint
@Table(uniqueConstraints = @UniqueConstraint(
```

```
    columnNames = {"customer_id", "invoice_month"}
))
```

## 20.3 Deadlocks in Batch

**Problem:** Multiple batch threads deadlock on database rows.

**Solution:**

```
// 1. Process disjoint data ranges (partitioning)
// 2. Consistent lock ordering
// 3. Reduce transaction scope (smaller chunks)
// 4. Use optimistic locking
@Version
private Long version; // JPA optimistic lock

// 5. Retry on deadlock
.retry(DeadlockLoserDataAccessException.class)
.retryLimit(3)
```

## 20.4 Memory Leaks

**Problem:** Job processes millions of records, JVM runs out of memory.

**Solution:**

```
// 1. Use paging reader (not cursor for huge datasets)
// 2. Clear JPA session periodically
@Bean
public JpaPagingItemReader<Entity> reader(EntityManagerFactory emf) {
    JpaPagingItemReader<Entity> reader = new JpaPagingItemReaderBuilder<Entity>
()
        .entityManagerFactory(emf)
        .queryString("SELECT e FROM Entity e")
        .pageSize(200)
        .build();
    // Clear persistence context after each page read
    return reader;
}

// 3. Use stateless session (Hibernate)
```

```
// 4. Process in smaller batches
// 5. Set appropriate JVM heap: -Xmx2g -Xms1g
```

## 20.5 Thread Starvation

**Problem:** Scheduled tasks pile up because default scheduler has only 1 thread.

**Solution:**

```java
@Configuration
public class SchedulerConfig implements SchedulingConfigurer {
    @Override
    public void configureTasks(ScheduledTaskRegistrar registrar) {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(10); // 10 threads for scheduled tasks
        scheduler.initialize();
        registrar.setTaskScheduler(scheduler);
    }
}
```

---

# Part 21: Interview Questions (50+)

## Spring Scheduler Questions

**Q1. What annotations are needed to enable scheduling in Spring Boot?** `@EnableScheduling` on a configuration class and `@Scheduled` on the methods to schedule. The scheduled class must be a Spring bean ( `@Component` ).

**Q2. What is the difference between fixedRate and fixedDelay?** `fixedRate` starts the timer from the **beginning** of the previous execution, so tasks may overlap if execution takes longer than the rate. `fixedDelay` starts the timer from the **end** of the previous execution, guaranteeing no overlap.

**Q3. What is the default thread pool size for @Scheduled? One (1) thread.** This means all scheduled tasks share a single thread and run sequentially. Configure via `SchedulingConfigurer` to increase.

**Q4. How do you externalize cron expressions?** Use SpEL: `@Scheduled(cron = "${app.cron.cleanup}")` and define the cron string in `application.yml` .

**Q5. What happens if a scheduled task throws an exception?** The exception is logged but does NOT stop future executions. The scheduler continues to fire at the next scheduled time. Use `ErrorHandler` for custom handling.

**Q6. How do you disable a scheduled task in a specific environment?** Use `@ConditionalOnProperty` : `@ConditionalOnProperty(name = "scheduler.enabled", havingValue = "true")` or use `-` as cron: `@Scheduled(cron = "${job.cron:-}")` .

**Q7. Can two scheduled methods run at the same time?** Not by default (single thread). With a configured thread pool of size > 1, yes, different @Scheduled methods can run concurrently.

**Q8. How do you handle scheduling in a clustered environment?** Use ShedLock or Quartz with JDBC JobStore. ShedLock uses a database lock table to ensure only one instance executes the job.

## Spring Batch Questions

**Q9. What is the difference between a Job and a Step?** A Job is the overall batch process containing one or more Steps. A Step is a single, independent phase of a Job that has its own reader, processor, and writer.

**Q10. What is chunk-oriented processing?** Data is read, processed, and written in groups (chunks). A chunk of N items is read one at a time, processed, then all N items are written together in a single transaction.

**Q11. What happens if a chunk fails?** The entire chunk transaction is rolled back. With skip policy, individual failed items can be skipped and the rest of the chunk retried.

**Q12. What is the role of JobRepository?** Stores metadata about job/step executions in database tables (BATCH_JOB_INSTANCE, BATCH_JOB_EXECUTION, BATCH_STEP_EXECUTION). Enables restart, monitoring, and auditing.

**Q13. How does restart work in Spring Batch?** On restart, Spring Batch reads the last successful commit count from BATCH_STEP_EXECUTION and resumes from that point. Paging readers track their position for restart.

**Q14. What is the difference between JdbcPagingItemReader and JpaPagingItemReader?** JdbcPagingItemReader uses raw JDBC with SQL and supports restart natively. JpaPagingItemReader uses JPA/JPQL but has overhead from persistence context management.

**Q15. What is a Tasklet step?** A step that executes a single task (not chunk-oriented). Used for simple operations like deleting files, running a stored procedure, or sending a notification.

**Q16. What is skip logic in Spring Batch?** Configures the job to skip (ignore) items that cause certain exceptions, up to a configurable limit. Skipped items are logged but don't fail the job.

**Q17. What is retry logic?** Automatically retries a failed item operation a configurable number of times before skipping or failing. Useful for transient errors like database timeouts.

**Q18. How do you pass data between steps?** Use `ExecutionContext` -- a key-value map available per Step or per Job. Step-level context is isolated; Job-level context is shared across steps.

**Q19. What is partitioning in Spring Batch?** Splitting the input data into independent partitions (ranges) and processing each partition in a separate thread. Significantly improves throughput for large datasets.

**Q20. What is the throttle limit?** Controls the maximum number of concurrent threads in a multi-threaded step. `throttleLimit(8)` means at most 8 threads process chunks simultaneously.

## Advanced Questions

**Q21. How do you trigger a batch job via REST API?** Inject `JobLauncher` and `Job` beans, create `JobParameters`, and call `jobLauncher.run(job, params)` from a controller method.

**Q22. How is Spring Batch different from Quartz?** Spring Batch is a **processing framework** (read, process, write data). Quartz is a **scheduling framework** (trigger jobs at times). They complement each

other -- Quartz schedules, Spring Batch processes.

**Q23. What are the batch job statuses?** STARTING, STARTED, STOPPING, STOPPED, FAILED, COMPLETED, ABANDONED, UNKNOWN.

**Q24. How do you make a batch job idempotent?** Check for existing processed records before creating new ones. Use unique constraints. Use database upsert operations. Make job parameters date-based.

**Q25. How to handle millions of records efficiently?** Chunk processing, paging readers, partitioning, multi-threaded steps, indexed DB queries, batch writes, and appropriate JVM memory.

**Q26. What is the difference between multi-threaded step and partitioning?** Multi-threaded step: multiple threads share the same reader (must be thread-safe). Partitioning: each thread has its own reader working on a disjoint data range (preferred).

**Q27. How do you unit test a batch job?** Use `@SpringBatchTest` which provides `JobLauncherTestUtils`. Assert on `JobExecution.getStatus()`, step read/write counts.

**Q28. What is flow-based job?** A job with conditional step execution: `on("COMPLETED").to(nextStep).on("FAILED").to(errorStep)`.

**Q29. How do you handle job scheduling in Kubernetes?** Use CronJob resources in Kubernetes instead of `@Scheduled`. Each CronJob creates a Pod that runs the batch job and terminates.

**Q30. What is remote chunking?** Processing is distributed across multiple JVMs. The master reads and sends items to worker JVMs which process and write. Items are sent via messaging (JMS, RabbitMQ).

## Architect-Level Questions

**Q31. Design a batch system for processing 10 million records nightly.** Use partitioning (8-16 partitions by ID range), JdbcPagingItemReader, batch size 200, separate IO and CPU thread pools, retry on transient errors, skip on data errors, monitoring with metrics.

**Q32. How to ensure exactly-once processing in batch?** Unique constraints + idempotent operations + partition by non-overlapping ranges + transactional chunk processing.

**Q33. How to handle batch in a cloud-native microservices environment?** Separate batch processing service, triggered via scheduler or Kafka events, stores execution metadata in its own database, publishes completion events to Kafka.

**Q34. Design a file-based batch processing system.** FlatFileItemReader with line tokenizer -> validation processor -> JPA writer. Handle multiple file formats, bad record files, completion flags.

**Q35. How to implement a batch job that calls external APIs?** Custom ItemProcessor with WebClient/RestTemplate. Handle rate limiting, circuit breaker (Resilience4j), retry on 5xx errors, skip on 4xx errors, timeout configuration.

**Q36. What are the tradeoffs of chunk size?** Small chunks: more transactions, fine-grained restart, slower. Large chunks: fewer transactions, coarse restart, faster but more memory. Typical production: 100-500.

**Q37. How to implement conditional step execution?**

```
.start(validationStep)
    .on("COMPLETED").to(processingStep)
    .on("FAILED").to(errorNotificationStep)
    .end()
```

**Q38. How to prevent concurrent execution of the same batch job?** Use `JobParametersIncrementer` with unique run IDs. Or use ShedLock for distributed locking. Or check `JobExplorer` for running executions before launching.

**Q39. Design a batch system with rollback and compensating transactions.** Use savepoints per chunk. On failure, execute compensating step that reverses partial changes. Log all operations for audit trail.

**Q40. How to implement event-driven batch triggering?** Kafka consumer listens for trigger events -> calls `JobLauncher.run()`. Decouple scheduling from batch execution.

**Q41. What is the maximum throughput you can achieve with Spring Batch?** Depends on infrastructure. With 8 partitions, batch writes, indexed queries: 50,000-100,000 records/minute is typical. Remote chunking can scale further.

**Q42. How to handle timezone issues in scheduled jobs?** Use `@Scheduled(cron = "...", zone = "America/New_York")`. Store all timestamps in UTC internally, convert for display.

**Q43. Design a retry strategy for a batch job calling a flaky external API.** Exponential backoff: 1s, 2s, 4s. Circuit breaker after 5 consecutive failures. Dead letter queue for items that exhaust retries. Manual intervention for dead-lettered items.

**Q44. How to version and migrate batch job configurations?** Use Flyway/Liquibase for batch metadata tables. Version job configurations alongside application code. Never modify completed job parameters.

**Q45. How to implement a data migration batch job between two databases?** JdbcCursorItemReader from source DB -> transformation processor -> JdbcBatchItemWriter to target DB. Partitioned by table/date range. Validation step after migration.

**Q46. How to test batch jobs with large datasets?** Integration tests with H2/embedded DB. Subset testing (1% of production data). Performance tests in staging environment. Property-based testing for processors.

**Q47. How do you handle schema changes in batch tables?** Spring Batch schema changes are rare. Use Flyway migration scripts. Test with both old and new schema versions.

**Q48. What is the difference between Spring Batch and Apache Spark?** Spring Batch: single-JVM, chunk processing, enterprise integration. Spark: distributed computing, in-memory, big data analytics. Use Batch for < 100M records, Spark for bigger.

**Q49. How to implement audit logging in batch jobs?** StepExecutionListener logs step metrics. Custom ItemWriteListener logs each batch. ExecutionContext stores processing metadata.

**Q50. How to implement batch job SLAs?** Monitor job duration via metrics. Alert if job exceeds SLA threshold. Implement kill switch for long-running jobs. Auto-scale partitions based on data volume.

**Q51. How to decommission a batch job safely?** Disable via property flag. Monitor for any dependent systems. Keep job definition for restart of failed historical runs. Archive batch metadata after retention period.

**Q52. How to handle daylight saving time in cron expressions?** Spring handles DST transitions. For critical midnight jobs, schedule at 1 AM instead of midnight. Always specify timezone in the cron annotation.

---

# Part 22: Coding Exercises (15+)

## Exercise 1: Simple Fixed-Rate Scheduler

```java
@Component
public class HeartbeatScheduler {
    private final AtomicInteger count = new AtomicInteger(0);

    @Scheduled(fixedRate = 5000)
    public void heartbeat() {
        System.out.printf("[%s] Heartbeat #%d on thread %s%n",
            Instant.now(), count.incrementAndGet(),
            Thread.currentThread().getName());
    }
}
```

## Exercise 2: Cron-Based Report Scheduler

```java
@Component
public class DailyReportScheduler {

    @Autowired private OrderRepository orderRepository;

    @Scheduled(cron = "0 0 7 * * MON-FRI")
    public void generateDailyReport() {
        LocalDate yesterday = LocalDate.now().minusDays(1);
        List<Order> orders = orderRepository.findByOrderDate(yesterday);

        long totalOrders = orders.size();
        BigDecimal totalRevenue = orders.stream()
            .map(Order::getAmount)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        System.out.printf("Daily Report for %s: Orders=%d, Revenue=$%s%n",
            yesterday, totalOrders, totalRevenue);
    }
}
```

## Exercise 3: Configurable Scheduler with Thread Pool

```java
@Configuration
@EnableScheduling
public class SchedulerConfig implements SchedulingConfigurer {

    @Override
    public void configureTasks(ScheduledTaskRegistrar registrar) {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5);
        scheduler.setThreadNamePrefix("my-scheduler-");
        scheduler.setErrorHandler(t ->
            System.err.println("Task error: " + t.getMessage()));
        scheduler.initialize();
        registrar.setTaskScheduler(scheduler);
    }
}
```

## Exercise 4: Basic Spring Batch Job (CSV to Database)

```java
@Configuration
public class CsvImportJobConfig {

    @Bean
    public FlatFileItemReader<Product> csvReader() {
        return new FlatFileItemReaderBuilder<Product>()
            .name("productCsvReader")
            .resource(new ClassPathResource("products.csv"))
            .delimited()
            .names("name", "category", "price")
            .targetType(Product.class)
            .build();
    }

    @Bean
    public ItemProcessor<Product, Product> priceValidator() {
        return product -> {
            if (product.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
                return null; // Skip invalid prices
            }
            product.setImportDate(LocalDate.now());
```

```
            return product;
        };
    }

    @Bean
    public JpaItemWriter<Product> productWriter(EntityManagerFactory emf) {
        JpaItemWriter<Product> writer = new JpaItemWriter<>();
        writer.setEntityManagerFactory(emf);
        return writer;
    }

    @Bean
    public Job importJob(JobRepository repo, Step importStep) {
        return new JobBuilder("csvImportJob", repo)
            .start(importStep).build();
    }

    @Bean
    public Step importStep(JobRepository repo,
                           PlatformTransactionManager tx) {
        return new StepBuilder("importStep", repo)
            .<Product, Product>chunk(50, tx)
            .reader(csvReader())
            .processor(priceValidator())
            .writer(productWriter(null))
            .build();
    }
}
```

## Exercise 5: Batch Job with Skip and Retry

```
@Bean
public Step faultTolerantStep(JobRepository repo,
                              PlatformTransactionManager tx) {
    return new StepBuilder("faultTolerant", repo)
        .<Customer, Invoice>chunk(100, tx)
        .reader(customerReader())
        .processor(invoiceProcessor())
        .writer(invoiceWriter())
        .faultTolerant()
        .skipLimit(20)
        .skip(DataFormatException.class)
        .retryLimit(3)
        .retry(TransientDataAccessException.class)
```

```
        .listener(new SkipListener<Customer, Invoice>() {
            public void onSkipInProcess(Customer c, Throwable t) {
                System.err.println("Skipped customer " + c.getId());
            }
        })
        .build();
}
```

## Exercise 6: Multi-Step Job (Process then Notify)

```
@Bean
public Job multiStepJob(JobRepository repo,
                        Step processStep, Step notifyStep) {
    return new JobBuilder("multiStepJob", repo)
        .start(processStep)
        .next(notifyStep)
        .build();
}

@Bean
public Step notifyStep(JobRepository repo,
                       PlatformTransactionManager tx) {
    return new StepBuilder("notifyStep", repo)
        .tasklet((contribution, chunkContext) -> {
            int processed = chunkContext.getStepContext()
                .getStepExecution().getJobExecution()
                .getExecutionContext().getInt("processedCount", 0);
            emailService.send("admin@company.com",
                "Batch complete: " + processed + " records");
            return RepeatStatus.FINISHED;
        }, tx).build();
}
```

## Exercise 7: Partitioned Batch Job

```
@Bean
public Step masterStep(JobRepository repo, Step workerStep) {
    return new StepBuilder("masterStep", repo)
        .partitioner("workerStep", idRangePartitioner())
        .step(workerStep)
```

```
        .gridSize(4) // 4 partitions
        .taskExecutor(new SimpleAsyncTaskExecutor())
        .build();
}

@Bean
public Partitioner idRangePartitioner() {
    return gridSize -> {
        Map<String, ExecutionContext> map = new HashMap<>();
        long total = customerRepository.count();
        long range = total / gridSize + 1;
        for (int i = 0; i < gridSize; i++) {
            ExecutionContext ctx = new ExecutionContext();
            ctx.putLong("minId", i * range + 1);
            ctx.putLong("maxId", (i + 1) * range);
            map.put("partition" + i, ctx);
        }
        return map;
    };
}
```

## Exercise 8: Scheduler-Triggered Batch Job

```
@Component
public class NightlyBatchTrigger {

    @Autowired private JobLauncher jobLauncher;
    @Autowired private Job processJob;

    @Scheduled(cron = "0 0 0 * * *")
    public void triggerNightlyJob() {
        try {
            JobParameters params = new JobParametersBuilder()
                .addLocalDate("runDate", LocalDate.now())
                .addLong("runId", System.currentTimeMillis())
                .toJobParameters();
            JobExecution ex = jobLauncher.run(processJob, params);
            System.out.println("Job status: " + ex.getStatus());
        } catch (Exception e) {
            System.err.println("Job failed: " + e.getMessage());
        }
```

```
    }
}
```

## Exercise 9: REST API to Trigger and Monitor Batch

```
@RestController
@RequestMapping("/api/batch")
public class BatchController {

    @Autowired private JobLauncher launcher;
    @Autowired private Job job;
    @Autowired private JobExplorer explorer;

    @PostMapping("/run")
    public ResponseEntity<Map<String, Object>> runJob() throws Exception {
        JobParameters params = new JobParametersBuilder()
            .addLong("time", System.currentTimeMillis())
            .toJobParameters();
        JobExecution exec = launcher.run(job, params);
        return ResponseEntity.ok(Map.of(
            "jobId", exec.getId(),
            "status", exec.getStatus().toString()
        ));
    }

    @GetMapping("/status/{jobId}")
    public ResponseEntity<Map<String, Object>> getStatus(
            @PathVariable Long jobId) {
        JobExecution exec = explorer.getJobExecution(jobId);
        return ResponseEntity.ok(Map.of(
            "status", exec.getStatus().toString(),
            "startTime", exec.getStartTime().toString(),
            "endTime", exec.getEndTime() != null ?
                exec.getEndTime().toString() : "Running"
        ));
    }
}
```

## Exercise 10: Conditional Flow Job

```java
@Bean
public Job conditionalJob(JobRepository repo,
                          Step validation, Step process, Step error) {
    return new JobBuilder("conditionalJob", repo)
        .start(validation)
            .on("COMPLETED").to(process)
            .from(validation)
            .on("FAILED").to(error)
        .end()
        .build();
}
```

## Exercise 11: Database to CSV Export

```java
@Bean
public FlatFileItemWriter<Order> csvExportWriter() {
    return new FlatFileItemWriterBuilder<Order>()
        .name("orderCsvWriter")
        .resource(new FileSystemResource("export/orders.csv"))
        .delimited()
        .delimiter(",")
        .names("id", "customerName", "amount", "date")
        .headerCallback(writer ->
            writer.write("ID,Customer,Amount,Date"))
        .build();
}
```

## Exercise 12: Custom ItemReader from REST API

```java
@Component
@StepScope
public class ApiItemReader implements ItemReader<ExternalData> {

    private List<ExternalData> data;
    private int index = 0;

    @BeforeStep
    public void init(StepExecution stepExecution) {
```

```
        this.data = restTemplate.exchange(
            "https://api.example.com/data",
            HttpMethod.GET, null,
            new ParameterizedTypeReference<List<ExternalData>>() {}
        ).getBody();
    }

    @Override
    public ExternalData read() {
        if (index < data.size()) {
            return data.get(index++);
        }
        return null; // No more items
    }
}
```

## Exercise 13: ShedLock for Distributed Scheduling

```
// Dependency: net.javacrumbs.shedlock:shedlock-spring

@Configuration
@EnableSchedulerLock(defaultLockAtMostFor = "10m")
public class ShedLockConfig {
    @Bean
    public LockProvider lockProvider(DataSource dataSource) {
        return new JdbcTemplateLockProvider(dataSource);
    }
}

@Component
public class DistributedScheduler {

    @Scheduled(cron = "0 0 * * * *")
    @SchedulerLock(name = "hourlySync",
                   lockAtMostFor = "55m",
                   lockAtLeastFor = "5m")
    public void syncData() {
        // Only ONE instance runs this across the cluster
        dataService.syncAll();
    }
}
```

## Exercise 14: Job Execution Listener with Metrics

```java
@Component
public class MetricsJobListener implements JobExecutionListener {
    @Autowired private MeterRegistry registry;

    @Override
    public void beforeJob(JobExecution exec) {
        registry.counter("batch.jobs.started",
            "job", exec.getJobInstance().getJobName()).increment();
    }

    @Override
    public void afterJob(JobExecution exec) {
        String job = exec.getJobInstance().getJobName();
        registry.counter("batch.jobs.completed",
            "job", job,
            "status", exec.getStatus().toString()).increment();
    }
}
```

## Exercise 15: Composite Processor (Chain Multiple Processors)

```java
@Bean
public CompositeItemProcessor<RawData, ProcessedData> compositeProcessor() {
    CompositeItemProcessor<RawData, ProcessedData> processor =
        new CompositeItemProcessor<>();
    processor.setDelegates(List.of(
        validationProcessor(),
        transformationProcessor(),
        enrichmentProcessor()
    ));
    return processor;
}
```

---

*End of Guide*

**Document Version:** 1.0 **Last Updated:** March 2026 **Topics Covered:** 22 Parts, 52 Interview Questions, 15 Coding Exercises